

第四讲 - 元组与字符串 (编程练习 1-2-3)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



- 1 从列表到不可变序列
- 2 序列的共同操作
- 3 元组：固定结构的数据
- 4 字符串：文本数据
- 5 字符串清洗与拆分
- 6 字符串格式化
- 7 综合实践
- 8 总结与课后拓展



1 为什么学元组和字符串?

- 字符串: 接收、清理和拆分文本
- 元组: 保存结构固定的数据记录
- 业务例子: 原始订单文本 → 清洗后订单记录

```
raw_order = "ORD-20260928-001,泰国,299.90,PAID"
```



```
order = ("ORD-20260928-001", "泰国", 299.90, "PAID")
```



字符串管文本, 元组管固定结构。

5 字符串: 文本数据

- 用单引号或双引号创建字符串
- 字符串也支持索引、切片、len()、in
- 字符串不可变, 修改其实是得到新字符串

```
status = "paid"
cleaned_status = status.upper() # "PAID"
```

- 从订单编号中提取信息:

```
order_id = "ORD-20260928-001"
prefix = order_id[:3] # "ORD"
order_date = order_id[4:12] # "20260928"
serial_number = order_id[13:] # "001"
```

虽然字符串不能修改, 但我们可以生成新的字符串!

2 序列的共同操作

列表、元组、字符串都是“有序序列”

操作	说明
索引	取一个元素, 如 s[0]
切片	取一段内容 (含start, 不含stop)
len()	看长度
in	判断是否包含
+	连接
*	重复

三种序列的对比

列表

可改

[1, 2, 3]

元组

不可改

(1, 2, 3)

字符串

不可改

"abc"

6 字符串清洗与拆分

strip()	删首尾空白
upper() / lower()	统一大小写
replace(a, b)	替换内容
split()	拆成列表
join()	把多个字符串连回文本

常用流程:

```
raw_order = "ORD-20260928-001,泰国,299.90,PAID"
cleaned_order = raw_order.strip()
fields = cleaned_order.split(",")
```

```
order_id = fields[0].strip()
market = fields[1].strip()
amount = float(fields[2].strip())
status = fields[3].strip().upper()
```

文本先清洗, 再拆分, 再转换。

3 元组: 固定结构的数据

- 创建元组:

```
order = ("ORD-20260928-001", "泰国", 299.90, "PAID")
```
- 单元素元组要加逗号:

```
("PAID",)
```
- 也可以用 tuple() 转换:

```
tuple(["泰国", "越南"]) → ("泰国", "越南")
```
- 元组不可直接修改
- 适合保存: 订单快照、坐标、固定字段记录

不能像列表那样直接改元素!



4 元组打包与解包

```
order = ("ORD-20260928-001", "泰国", 299.90, "PAID")
```

解包: 按顺序拆给不同变量

```
order_id, market, amount, status = order
```

order_id
ORD-20260928-001

market
泰国

amount
299.90

status
PAID

- 打包: 多个值组合成一个元组
- 解包: 按顺序拆给不同变量

左右数量要一致。

一个元组
就是一条订单记录!



7 业务文本格式化

- 用 f-string 让字符串更易读:

```
message = f"订单{order_id} | 市场:{market} | 金额:{amount:.2f}元 | 状态:{status}"
```
- f"...{变量}..." 把数据放进文字里
- .2f: 保留两位小数
- .,2f: 千位分隔符 + 两位小数
- .1%: 显示百分比
- 输出不只是“能看见”, 还要“看得清”。

格式化效果示例:

```
amount = 1234567.8
f"{amount:,.2f}" # 1,234,567.80
rate = 0.1234
f"{rate:.1%}" # 12.3%
```



8 综合实践: 跨境订单摘要

原始订单文本:

```
raw_order = "ORD-20260928-001 | 泰国 | 无线耳机 | 29.90 | 8 | 0.10 | paid"
```

- 1 清理文本 去除首尾空白
- 2 拆分字段 按 | 拆分, 得到各个字段
- 3 类型转换与计算 计算商品原价、优惠金额、应付金额
- 4 生成摘要 输出清晰的订单信息

计算结果:

```
商品原价: 239.20 元
优惠金额: 23.92 元
应付金额: 215.28 元
订单状态: PAID
```

从一行文本
到清晰的业务摘要!



记住: 元组适合存“固定结构”, 字符串适合处理“文本信息”。

先清洗, 再拆分, 再转换, 再输出。

本讲运行示例：整理跨境订单记录

业务输入

外部系统传来一条订单文本，其中包含订单编号、市场、订单金额和订单状态。

```
raw_order = "  ORD-20260928-001,泰国,299.90,paid  "
```

原始文本需要处理

- 两端存在多余空格；
- 多个字段挤在一个字符串中；
- 金额仍然是文本；
- 订单状态的格式不统一。

两种数据类型的分工

字符串用于接收、清理和拆分文本。**元组**用于保存字段顺序固定、不希望被随意修改的订单记录。

期望得到的订单记录

清理并转换数据后，用一个**结构固定**的**元组**保存订单。

```
order = (  
    "ORD-20260928-001",  
    "泰国",  
    299.90,  
    "PAID"  
)
```

列表、元组与字符串的联系

类型	示例	元素	可修改	典型用途
列表	["泰国", "越南"]	多个数据	可以	维护可能变化的数据集
元组	("订单001", 299.9)	多个数据	不可以	保存结构固定的数据记录
字符串	"泰国"	多个字符	不可以	保存和处理文本数据

共同点：都是序列

列表、元组和字符串都按照确定的顺序保存元素，因此都支持索引、切片、`len()` 和 `in` 等序列操作。

主要区别：能否直接修改

列表创建后仍然可以增加、删除或修改元素。元组和字符串属于不可变序列，不能通过索引直接修改其中的元素；需要改变内容时，Python会产生一个新的元组或字符串。

本讲总览：先把列表中已经掌握的序列操作迁移到元组和字符串，再重点学习元组的固定结构以及字符串的文本处理能力。

索引与切片

```
markets_list = ["泰国", "越南", "新加坡"]
markets_tuple = ("泰国", "越南", "新加坡")
market_text = "泰国越南新加坡"
```

索引：取得一个元素

三种序列的索引都从 0 开始，也都支持负索引。

```
print(markets_list[0]) # 泰国
print(markets_tuple[0]) # 泰国
print(market_text[0]) # 泰
print(market_text[-1]) # 坡
```

切片：取得一个子序列

切片仍然遵循“包含起点、不包含终点”的规则。

```
print(markets_list[0:2])
# ['泰国', '越南']

print(markets_tuple[0:2])
# ('泰国', '越南')

print(market_text[0:2]) # 泰国
```

注意：列表和元组中的一个元素可以是完整的市场名称。字符串中的一个元素是一个字符。**切片返回与原序列类型相同的新序列。**

长度、成员判断、连接与重复

元组

长度表示元素数量，成员判断检查一个完整元素是否存在。

```
markets = ("泰国", "越南")
print(len(markets)) # 2
print("越南" in markets) # True
extended_markets = markets + (
    "新加坡", "菲律宾"
)
repeated_markets = markets * 2
```

字符串

长度表示字符数量，成员判断可以检查字符或子字符串。

```
market_text = "泰国市场"
print(len(market_text)) # 4
print("泰国" in market_text)
# True
report_title = (
    market_text + "周报"
)
separator = "=" * 10
```

操作结果

`len()` 返回整数，`in` 返回布尔值。`+` 连接两个同类型序列，`*` 按照指定次数重复序列。对元组和字符串使用连接或重复操作时，Python会生成新对象，原序列保持不变。

索引、切片、长度、成员判断、连接和重复是序列的通用操作。

创建元组

直接创建元组

元组通常使用圆括号表示，各元素之间使用逗号分隔。

```
order = (  
    "ORD-20260928-001",  
    "泰国",  
    299.90,  
    "PAID"  
)  
  
empty_tuple = ()  
  
print(order)  
print(empty_tuple)
```

单元素元组

只有一个元素时，必须保留元素后面的逗号。

```
status_tuple = ("PAID",)  
not_a_tuple = ("PAID")  
  
print(type(status_tuple))  
# <class 'tuple'>  
  
print(type(not_a_tuple))  
# <class 'str'>
```

使用tuple()转换数据

`tuple(["泰国", "越南"])` 可以把列表转换为元组，结果为 `("泰国", "越南")`。

元组的不可变性

```
order = ("ORD-20260928-001", "泰国", 299.90, "PAID")
```

不能直接修改元素

元组不支持通过索引重新赋值。

```
order[2] = 329.90
```

```
# TypeError:  
# 'tuple' object does not  
# support item assignment
```

需要修改时创建新元组

根据原记录中的字段创建一个新的订单记录。

```
updated_order = (order[0],  
                 ↪ order[1], 329.90, order[3])  
  
print(order)  
print(updated_order)
```

为什么使用元组

不可变性可以减少订单编号、金额和状态被意外修改的风险。如果数据需要频繁增加、删除或调整，列表通常更加合适。

元组没有`append()`、`remove()`或`sort()`等修改方法。常用的`count()`和`index()`只查询数据，不会修改元组。

元组的打包与解包

打包：把多个值组合成元组

用逗号分隔多个值时，Python会把这些值组合成一个元组。

```
order = ("ORD-20260928-001", "泰国", 299.90, "PAID")  
  
print(order)
```

解包：把各元素赋给不同变量

按照元组中元素的顺序，为每个元素指定一个含义清楚的变量名。

```
order_id, market, amount, status = order  
print(order_id)  # ORD-20260928-001  
print(market)    # 泰国  
print(amount)    # 299.9  
print(status)    # PAID
```

普通解包时，左侧变量数量必须与元组中的元素数量一致，否则会产生 `ValueError`。

业务实例：保存订单快照

已经确认的订单数据

完成文本清理和类型转换后，把订单字段保存为一个结构固定的元组。

```
order = ("ORD-20260928-001", "泰国", 299.90, "PAID")

order_id, market, amount, status = order

print(
    f"订单号: {order_id}, "
    f"市场: {market}, "
    f"金额: ${amount:.2f}, "
    f"状态: {status}"
)
```

元组在这里解决的问题

订单记录的字段数量和排列顺序已经确定。元组能够把这些字段作为一个整体保存，解包后又可以通过有业务含义的变量名使用各个字段。**元组适合较短、结构固定并且主要用于读取的记录。**字段较多或需要根据字段名称访问数据时，后续学习的字典会更加清晰。

创建字符串

单引号与双引号

单引号和双引号没有功能差别，只要字符串两端的引号保持一致。

```
market_1 = '泰国'  
market_2 = "越南"  
  
message = '订单状态: "已支付"'  
  
empty_text = ""
```

多行字符串

三重引号可以创建**跨越多行并保留换行格式的字符串**。

```
order_note = """订单备注:  
客户要求周五前发货。  
商品需要加固包装。"""  
  
print(order_note)
```

转换数据类型

`str()` 可以把整数、浮点数等数据转换为字符串。例如，`str(128)` 的结果是字符串 "128"。字符串中的数字字符不能直接参加数值计算。例如，"299.90" 是文本，需要使用 `float()` 转换后才能作为金额计算。

从订单编号中提取信息

订单编号的固定结构

订单编号 ORD-20260928-001 依次包含业务前缀、日期和流水号。

```
order_id = "ORD-20260928-001"

prefix = order_id[:3]
order_date = order_id[4:12]
serial_number = order_id[13:]

print(prefix)           # ORD
print(order_date)       # 20260928
print(serial_number)    # 001
```

从订单编号中提取信息（续）

索引与切片

索引取得一个字符，切片取得一个新的字符串。负索引可以从字符串末尾开始定位。

```
print(order_id[0])      # 0
print(order_id[-1])     # 1
print(order_id[-3:])    # 001
```

长度与成员判断

`len()` 统计字符数量，`in` 可以检查子字符串是否存在。

```
print(len(order_id)) # 16

print("20260928" in order_id)
# True
```

使用切片的前提

固定位置切片适合格式稳定的编号。如果不同来源的文本格式不一致，应先清理或拆分文本，再提取字段。

字符串的不可变性

不能直接修改字符

字符串不支持通过索引重新赋值。

```
status = "paid"

status[0] = "p"

# TypeError:
# 'str' object does not
# support item assignment
```

不可变与重新赋值

不可变表示字符串中的字符不能原地修改。语句 `status = status.upper()` 可以执行，因为它让变量 `status` 重新指向一个新字符串。

方法返回新字符串

对字符串进行清理或转换时，Python 会返回一个新的字符串。

```
raw_status = " paid "

cleaned_status = (
    raw_status.strip().upper()
)

print(len(raw_status)) # 8
print(raw_status) # paid

print(len(cleaned_status)) # 4
print(cleaned_status) # PAID
```

只执行 `raw_status.strip()` 不会改变 `raw_status`。需要把方法返回的新字符串赋给变量，或者直接使用这个返回结果。

清理和统一文本格式

```
raw_status = " paid "  
status = raw_status.strip().upper()  
  
raw_sku = " sku 001-th "  
sku = (  
raw_sku.strip()  
.replace(" ", "")  
.upper()  
)  
  
print(status) # PAID  
print(sku) # SKU001-TH
```

常用的清洗方法

- `strip()` : 删除首尾空白;
- `upper()` : 转换为大写;
- `lower()` : 转换为小写;
- `replace(a, b)` : 把指定内容替换为新内容。

方法可以连续使用

字符串方法按照从左到右的顺序执行, 每一步都会返回一个新的字符串。

split(): 把一条文本拆成多个字段

```
raw_order = "   ORD-20260928-001,泰国,299.90,paid   "

cleaned_order = raw_order.strip()
fields = cleaned_order.split(",")

print(fields)
# ['ORD-20260928-001', '泰国', '299.90', 'paid']

print(type(fields)) # <class 'list'>
```

如何理解split()

- 逗号是字段之间的分隔符，拆分后不会保留；
- split() 返回一个列表，每个元素对应一个业务字段；
- 不指定分隔符时，split() 默认按照空白字符拆分。

从文本字段到订单记录

```
# 接续上一页的fields
order_id = fields[0].strip()
market = fields[1].strip()
amount = float(fields[2].strip())
status = fields[3].strip().upper()

order = (
    order_id,
    market,
    amount,
    status
)

print(order)
```

字段的业务含义

- `fields[0]` : 订单编号;
- `fields[1]` : 目标市场;
- `fields[2]` : 订单金额;
- `fields[3]` : 订单状态。

数据处理流程

原始文本 → 字段列表 → 类型转换
→ 订单元组。

join(): 把多个字符串连接成文本

```
tags = ["新品", "跨境电商", "泰国市场"]
tag_text = ",".join(tags)
```

```
print(tag_text)
# 新品、跨境电商、泰国市场
```

```
export_fields = [
    "ORD-20260928-001", "泰国", "299.90", "PAID"
]
export_line = ",".join(export_fields)
```

```
print(export_line)
# ORD-20260928-001,泰国,299.90,PAID
```

拆分与连接

`split()` 把字符串拆成列表； `join()` 把字符串序列连接成一个字符串。参与连接的元素必须都是字符串。

判断、查找与计数

```
order_text = "ORD-20260928-001,泰国,299.90,PAID"
```

```
print("PAID" in order_text)           # True
print(order_text.find("泰国"))        # 17
print(order_text.find("REFUND"))      # -1
print(order_text.count(","))          # 3
```

根据任务选择操作

- 只需判断是否存在：使用 `in`，结果为布尔值；
- 需要确定首次出现的位置：使用 `find()`；
- 需要统计出现次数：使用 `count()`。

注意

如果没有找到指定内容，`find()` 返回 `-1`。

验证输入文本

```
quantity_text = "128"  
market_text = "泰国"  
sku_text = "SKU001TH"  
amount_text = "299.90"  
  
print(quantity_text.isdigit()) # True  
print(market_text.isalpha()) # True  
print(sku_text.isalnum()) # True  
print(amount_text.isdigit()) # False, 因为其中包含小数点
```

常用的字符判断方法

- `isdigit()`：是否全部由数字字符组成；
- `isalpha()`：是否全部由文字或字母组成；
- `isalnum()`：是否只包含文字、字母或数字。

从“输出数据”到“传递信息”

```
order_id = "ORD-20260928-001"
market = "泰国"
amount = 299.9
status = "PAID"

print(order_id, market, amount, status)

message = (
    f"订单{order_id} | 市场: {market} | "
    f"金额: {amount}元 | 状态: {status}"
)
print(message)
```

两种输出的区别

直接输出变量适合检查程序；按照业务含义组织文本，更适合展示、沟通和生成报告。

文本格式化

把变量、计算结果和说明文字组合成结构清晰的字符串。

f-string: 把数据嵌入文本

```
product = "无线耳机"
unit_price = 29.90
quantity = 8
total = unit_price * quantity

message = (
    f"商品: {product}, "
    f"单价: {unit_price}元, "
    f"数量: {quantity}件, "
    f"合计: {total}元"
)

print(message)
```

基本写法和优点

在字符串前添加字母 **f**，再把变量或表达式写在花括号中。**优点是：**文字与变量的位置关系清晰；可以直接嵌入简单的计算表达式；不需要手动把数值转换为字符串。

按照业务习惯格式化数值

```
unit_price = 29.9
sales = 1234567.8
growth_rate = 0.186

print(f"商品单价: {unit_price:.2f} 元")
print(f"销售额: {sales:,.2f} 元")
print(f"同比增长: {growth_rate:.1%}")
```

常用的数值格式

- `.2f` : 保留两位小数;
- `,.2f` : 添加千位分隔符, 并保留两位小数;
- `.1%` : 转换为百分比, 并保留一位小数。

格式化不改变原始数值

格式说明只控制字符串中的显示效果, 变量本身仍然是原来的数值。

对齐文本与数值

```
print(f"{'SKU':<12}{'QTY':>8}{'SALES':>14}")  
print(f"{'SKU-A01':<12}{'128:>8'}{'3827.2:>14,.2f}")  
print(f"{'SKU-B08':<12}{'9:>8'}{'15200:>14,.2f}")
```

常用的对齐方式

- < : 左对齐, 常用于文本;
- > : 右对齐, 常用于数值;
- ^ : 居中对齐;
- 对齐符号后面的数字表示字段的最小宽度。

为什么数值通常右对齐

右对齐可以让个位、小数位和千位分隔符处于相近位置, 便于快速比较。

生成订单业务摘要

```
order = ("ORD-20260928-001", "泰国", 299.9, "PAID")

order_id, market, amount, status = order

summary = (
    "订单摘要\n"
    f"订单编号: {order_id}\n"
    f"目标市场: {market}\n"
    f"订单金额: {amount:,.2f} 元\n"
    f"订单状态: {status}"
)

print(summary)
```

完整的数据处理链

清洗原始文本 → 拆分并转换字段 → 组织结构化数据 → 生成可读的业务信息。

程序输出也需要设计

优秀的输出应当含义明确、格式一致，并能帮助使用者快速检查关键数据。

实践任务：跨境订单摘要

```
raw_order = (  
    " ORD-20260928-001 | 泰国 | 无线耳机 | "  
    "29.90 | 8 | 0.10 | paid "  
)
```

原始数据包含的字段

订单编号、目标市场、商品名称、商品单价、购买数量、折扣率和订单状态。

需要完成的任务

- 1 清理文本，并拆分成各个字段；
- 2 把单价、数量和折扣率转换为合适的数据类型；
- 3 计算商品原价、优惠金额和应付金额；
- 4 生成格式清晰的订单摘要。

计算规则

商品原价 = 单价 × 数量，优惠金额 = 商品原价 × 折扣率，应付金额 = 商品原价 - 优惠金额。

任务分解与处理顺序

分解

把完整任务拆成文本清洗、字段转换、金额计算和结果输出四个环节。

模式识别与抽象

每条订单使用相同的分隔符和字段顺序。处理时保留业务字段，忽略多余空格和大小写差异。

可执行步骤

- ❶ 使用 `strip()` 清理整条文本；
- ❷ 使用 `split()` 拆分字段，并分别清理；
- ❸ 使用 `int()` 和 `float()` 转换数据类型；
- ❹ 完成计算，再使用 `f-string` 生成摘要。

完成标准

程序能够正确提取字段、计算应付金额，并输出便于人工核验的结果。

步骤1: 清洗并提取字段

```
cleaned_order = raw_order.strip()
fields = cleaned_order.split("|")

order_id = fields[0].strip()
market = fields[1].strip()
product = fields[2].strip()
price_text = fields[3].strip()
quantity_text = fields[4].strip()
discount_text = fields[5].strip()
status = fields[6].strip().upper()

print(fields)
print(order_id, market, product, status)
```

为什么先保留为字符串

`split()` 得到的所有字段都是字符串。先完成提取和清洗，再根据业务含义转换数据类型，更容易检查每一步的结果。

步骤2：构造记录并完成计算

```
unit_price = float(price_text)
quantity = int(quantity_text)
discount_rate = float(discount_text)

order = (
    order_id, market, product,
    unit_price, quantity,
    discount_rate, status
)

subtotal = unit_price * quantity
discount_amount = subtotal * discount_rate
payable = subtotal - discount_amount

print(order)
print(subtotal, discount_amount, payable)
```

数据类型与业务含义

数量使用 `int`，单价、折扣率和金额使用 `float`；元组保存本次处理得到的订单记录。商品原价为 239.20 元，优惠金额为 23.92 元，应付金额为 215.28 元。

步骤3：生成业务摘要

```
summary = (  
    "跨境订单摘要\n"  
    f"订单编号: {order_id}\n"  
    f"目标市场: {market}\n"  
    f"商品: {product}\n"  
    f"数量: {quantity} 件\n"  
    f"商品原价: {subtotal:,.2f} 元\n"  
    f"优惠金额: {discount_amount:,.2f} 元\n"  
    f"应付金额: {payable:,.2f} 元\n"  
    f"订单状态: {status}"  
)  
  
print(summary)
```

输出检查

金额统一保留两位小数，订单状态统一为大写，各项业务信息分别显示，便于核对。

本例的数据处理过程

原始字符串 → 字段列表 → 订单元组 → 计算结果 → 业务摘要。

第二条订单的结果验证

```
raw_order = (  
    " ORD-20260929-002 | 印尼 | 蓝牙音箱 | "  
    "45.50 | 3 | 0.00 | shipped "  
)
```

课堂任务

保留原来的处理步骤，只替换 `raw_order`，重新运行代码并检查结果。

预期结果

- 商品原价： 136.50 元；
- 优惠金额： 0.00 元；
- 应付金额： 136.50 元；
- 订单状态： SHIPPED 。

本讲总结

- 列表、元组和字符串都属于序列，可以使用索引、切片、`len()` 和 `in`；
- 元组不可变，适合保存字段固定、处理完成后不应随意修改的数据；
- 字符串不可变，字符串方法返回处理后的新字符串；
- `strip()`、`split()` 和 `join()` 用于清洗、拆分和连接文本；
- `f-string` 可以把变量和计算结果转换为清晰的业务信息。

本讲的数据处理过程

原始文本 → 清洗和拆分 → 类型转换 → 结构化记录 → 业务计算 → 格式化输出。

学习重点

记住常用操作的同时，更重要的是能够根据数据含义选择类型，并把业务任务分解为可执行、可检查的步骤。

列表、元组与字符串的选择

类型	可修改	主要内容	典型业务场景
列表	是	一组数据	购物车商品、待处理订单、销售记录
元组	否	固定结构的数据	订单快照、坐标、函数返回的多个结果
字符串	否	文本数据	商品名称、订单编号、客户留言

选择数据类型的思路

- 数据需要增加、删除或修改时，通常使用列表；
- 数据结构固定，希望避免意外修改时，可以使用元组；
- 数据表达文字、编号或消息时，使用字符串。

数据类型可以相互配合

字符串可以拆成列表，列表中的字段经过转换后可以组成元组，元组中的数据又可以格式化为新的字符串。

课后实践：客户询价单

```
raw_inquiry = (  
    " INQ-20260929-018 | 越南 | 咖啡机 | "  
    "79.90 | 18 | 0.05 | pending "  
)
```

实践要求

- 1 清理文本，并按照竖线拆分字段；
- 2 转换单价、数量和折扣率的数据类型；
- 3 使用元组保存处理后的询价记录；
- 4 计算商品原价、优惠金额和报价金额；
- 5 使用 f-string 生成格式清晰的询价摘要。

用于核验的结果

商品原价为 1,438.20 元，优惠金额为 71.91 元，报价金额为 1,366.29 元，状态应转换为 PENDING 。

课后拓展：前缀与后缀判断

```
order_id = "ORD-20260928-001"
filename = "orders_202609.csv"

print(order_id.startswith("ORD-"))
# True

print(filename.endswith(".csv"))
# True

print(filename.endswith(("csv", "xlsx")))
# True
```

startswith()与endswith()

这两个方法分别判断字符串是否以指定内容开头或结尾，返回结果为 True 或 False 。

典型用途

检查订单编号的格式、判断数据文件类型，或者筛选具有特定命名规则的文本。多个备选前缀或后缀可以放在一个元组中。

课后拓展：控制拆分次数

```
note = "ORD-001:客户备注:17:00后送达"

order_id, separator, message = note.partition(":")

print(order_id)    # ORD-001
print(separator)   # :
print(message)     # 客户备注:17:00后送达

first, rest = note.split(":", 1)

print(first)       # ORD-001
print(rest)        # 客户备注:17:00后送达
```

两种处理方法

`partition()` 从第一个分隔符处分成三部分，并返回一个元组；`split(":", 1)` 最多拆分一次，并返回一个列表，当客户留言本身也包含冒号时，限制拆分次数可以保留后面的完整内容。

THE END